

COURSE

Основы программирования



JS

Классы и прототипы

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Особенности определения объектов в JavaScript. **3**

Атрибуты объекта Prototipe, Class, Extensible. **6**

Атрибут prototype 6

Атрибут class 6

Атрибут extensible 7

Понятие класса в JavaScript. **8**

Свойства и методы класса **9**

Открытые (public) свойства и методы 9

Закрытые (private) свойства и методы 10

Привилегированные (privileged) методы 11

Статические (static) члены 12

Замыкание (closure) **12**

Концепции ООП: инкапсуляция, наследование и полиморфизм. **13**

Инкапсуляция 13

Наследование 16

Полиморфизм 20

Литература и другие источники *22*

Вопросы для самопроверки *22*

ПЛАН ЛЕКЦИИ

1. Особенности определения объектов в JavaScript;
2. Атрибуты объекта Prototipe, Class, Extensible;
3. Понятие класса в JavaScript;
4. Свойства и методы класса;
5. Замыкание (closure);
6. Концепции ООП: инкапсуляция, наследование и полиморфизм.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

ОСОБЕННОСТИ ОПРЕДЕЛЕНИЯ ОБЪЕКТОВ В JAVASCRIPT.

Все в JavaScript, на самом деле, является объектом. ***Массив*** — это объект. ***Функция*** — это объект. ***Объект*** - тоже объект. Так что такое объект?

Объект - это коллекция свойств. Каждое свойство представляет собой пару имя-значение. Имя свойства - это строка, а значение свойства - строка, число, булево значение, или объект (включая массив и функцию).

Когда мы определяем какую-то переменную, например,

```
<!DOCTYPE HTML>
<html>
<head>
<script>
        var s = 'hello world';
        console.log(typeof s); // выводит string
</script>
</head>
<body />
<html>
```

мы, в действительности, неявным образом задаем свойство какого-то объекта. В данном случае, если рассматривать DOM структуру, таким объектом будет глобальный объект window:

```
console.log(s == window.s); // выводит true
console.log(typeof window); // выводит object
```

Более того, это свойство window.s само по себе является объектом, т.к. в нем уже изначально определена своя коллекция свойств:

```
console.log(s.length); // выводит 11 (число символов в строке)
```

При всем при том, что это, на первый взгляд, обычный строковый литерал!

Если значение свойства - функция, мы можем назвать это свойство ***методом объекта***. Чтобы вызвать метод объекта достаточно дописать после его имени две круглые скобки (). Когда метод объекта выполняется, переменная this внутри этой функции ссылается на сам объект. ***С помощью ключевого***

слова this метод объекта получает доступ ко всем остальным свойствам и методам объекта. Например,

```
var s = 'karamazov'; // создаем новое свойство s объекта window (window.s)
var f = function(){ // создаем новый метод f объекта window (window.f)
        console.log(this == window); // выводит true
        console.log(this.s); // выводит 'karamazov'
}
f(); // вызываем метод f объекта window (window.f())

var o = {}; // создаем новое свойство o объекта window (window.o)
o.s = 'karamazov'; // создаем новое свойство s объекта window.o (window.o.s)
o.f = function(){ // создаем новый метод f объекта window.o (window.o.f)
        console.log(this == o); // выводит true
        console.log(this.s); // выводит 'karamazov'
}
o.f(); // вызываем метод f объекта window.o (window.o.f())
```

Объект создается с помощью ***функции-конструктора***, инициализирующей объект, и ключевого слова ***new***. Функция-конструктор предоставляет те же возможности, что и класс в других языках программирования: а именно, описывает шаблон, по которому будут создаваться объекты (экземпляры) класса. В основе такого шаблона лежит перечисление свойств и методов, которыми будет обладать объект, созданный на основе данного класса.

Для всех встроенных типов данных в JavaScript существуют встроенные функции-конструкторы. Например, когда объявляется строковая переменная неявным образом вызывается встроенная функция-конструктор:

```
var str='karamazov';
var str = new String('karamazov');
```

Это же утверждение верно и для всех остальных типов данных JavaScript:

```
// число
var num = 12345.6789; // var num = new Number(12345.6789);
// булево значение
var bul = true; // var c = new Boolean(true);
// функция
var fun = function(x){var p = x}; // var fun = new Function('x', 'var p = x');
// массив
var arr = ['a', 'b', 'c']; // var arr = new Array('a', 'b', 'c');
// объект
var obj = {}; // var obj = new Object();
```

У всех этих объектов сразу же после создания определены все свойства и методы, описанные в их функциях-конструкторах (классах):

```
console.log(num.toFixed(1)); // выводит 12345.6
console.log(arr.length); // выводит 3
```

На самом деле, интерпретатор JavaScript действует несколько хитрее, чем может показаться из предыдущего примера. Так, несмотря на то, что следующий код показывает равенство двух переменных (объектов класса String), а при попытке определить новый пользовательский метод для str1 мы получим ошибку:

```
var str1 = 'karamazov';
var str2 = new String('karamazov');
console.log(str1 == str2); // выводит true

str1.tell = function(){
        console.log(this);
}
str1.tell(); // выводит ошибку 'str1.tell is not a function'
```

При этом, для str2 все сработает, как мы и ожидаем:

```
str2.tell = function(){
        console.log(this);
}
str2.tell(); // выводит 'karamazov'
```

Это ограничение, наложенное JavaScript на переменные (объекты), созданные через строковые, числовые и булевы литералы, тем не менее, не распространяется на объекты, созданные через литералы функции, массива или объекта. Т.е. переменным (объектам), содержащим в себе функцию, или массив, или объект можно напрямую присваивать пользовательские свойства и методы.

В следующем примере можно наглядно убедиться, что функция f, созданная как метод глобального объекта window, сама оказывается объектом, у которого могут быть свои собственные свойства и методы:

```
var s = 'futurico'; // создаем новое свойство s объекта window (window.s)
var f = function(){ // создаем новый метод f объекта window (window.f)
        console.log(this == window); // выводит true
        console.log(this.s); // выводит 'futurico'
}
f(); // вызываем метод f объекта window (window.f())

f.s = 'karamazov'; // создаем новое свойство s объекта window.f (window.f.s)
f.m = function(){ // создаем новый метод m объекта window.f (window.f.m)
        console.log(this == f); // выводит true
        console.log(this.s); // выводит 'karamazov'
```

```
}
f.m(); // вызываем метод m объекта window.f (window.f.m())
```

Атрибуты объекта Prototipe, Class, Extensible.

Все объекты имеют ***атрибуты prototype, class и extensible***.

Атрибут prototype

Атрибут ***prototype*** объекта определяет объект, от которого наследуются свойства.

Важно понимать, что, когда в программном коде встречается ссылка prototype, она обозначает обычное свойство объекта, а не атрибут prototype. Атрибут prototype устанавливается в момент создания объекта. Для объектов, созданных с помощью литералов, прототипом является ***Object.prototype***.

Прототипом объекта, созданного с помощью оператора new, является значение свойства prototype конструктора. А прототипом объекта, созданного с помощью ***Object.create()***, становится первый аргумент этой функции (который может иметь значение null).

Стандартом ECMAScript предусматривается возможность определить прототип любого объекта, если передать его ***методу Object.getPrototypeOf()***.

Объекты, созданные с помощью оператора new, обычно наследуют свойство constructor, ссылающееся на функцию-конструктор, использованную для создания объекта. И как уже говорилось выше, ***функции-конструкторы имеют свойство prototype***, которое определяет прототип объектов, созданных с помощью этого конструктора.

Обратите внимание, что объекты, созданные с помощью литералов объектов или Object.create(), получают свойство constructor, ссылающееся на конструктор Object(). Таким образом, constructor.prototype ссылается на истинный прототип для литералов объектов, но обычно это не так для объектов, созданных вызовом Object.create().

Чтобы определить, является ли один объект прототипом (или звеном в цепочке прототипов) другого объекта, следует использовать метод isPrototypeOf(). Чтобы узнать, является ли p прототипом obj, нужно записать выражение p.isPrototypeOf(obj). Например,

```
var p = {x:1};                          // Определить объект-прототип.
var obj = Object.create(p);             // Создать объект с этим прототипом.
p.isPrototypeOf(obj);                   // => true: obj наследует p
Object.prototype.isPrototypeOf(p);      // => true: p наследует Object.prototype
```

Атрибут class

Атрибут class - это строка, содержащая информацию о типе объекта. По умолчанию метод toString() (наследуемый от Object.prototype) возвращает строку вида:

[object class]

Поэтому, чтобы определить класс объекта, можно попробовать вызвать ***метод toString()*** этого объекта и извлечь из результата подстроку с восьмого по предпоследний символ. Вся хитрость состоит в том, что многие методы наследуют другие, более полезные реализации метода toString(), и чтобы вызвать нужную версию toString(), необходимо выполнить косвенный вызов с помощью метода ***Function.call()***.

В примере ниже определяется функция, возвращающая класс любого объекта, переданного ей:

```
// Название класса объекта
```

```
function classof(obj) {
    if (obj === null) return "Null";
    if (obj === undefined) return "Undefined";

    return Object.prototype.toString.call(obj).slice(8,-1);
}
```

Этой функции classof() можно передать любое значение, допустимое в языке JavaScript. Числа, строки и логические значения действуют подобно объектам, когда относительно них вызывается метод toString(), а значения null и undefined обрабатываются особо.

Атрибут extensible

Атрибут extensible объекта определяет, допускается ли добавлять в объект новые свойства. В ECMAScript все встроенные и определяемые пользователем объекты являются расширяемыми, если они не были преобразованы в нерасширяемые объекты, а расширяемость объектов среды выполнения по-прежнему определяется каждой конкретной реализацией.

Стандарт ECMAScript определяет функции для получения и изменения признака расширяемости объекта. Чтобы определить, допускается ли расширять объект, его следует передать ***методу Object.isExtensible()***.

Чтобы сделать объект нерасширяемым, его нужно передать ***методу Object.preventExtensions()***. Обратите внимание, что после того как объект будет сделан нерасширяемым, его нельзя снова сделать расширяемым. Отметьте также, что вызов preventExtensions() оказывает влияние только на расширяемость самого объекта. Если новые свойства добавить в прототип нерасширяемого объекта, нерасширяемый объект унаследует эти новые свойства.

Назначение атрибута extensible заключается в том, чтобы дать возможность «фиксировать» объекты в определенном состоянии, запретив внесение изменений. Атрибут объектов extensible часто используется совместно с атрибутами свойств configurable и writable, поэтому в ECMAScript определяются функции, упрощающие одновременную установку этих атрибутов.

Метод Object.seal() действует подобно методу Object.preventExtensions(), но он не только делает объект нерасширяемым, но и делает все свойства этого объекта недоступными для настройки. То есть в объект нельзя будет добавить новые свойства, а существующие свойства нельзя будет удалить или настроить. Однако существующие свойства, доступные для записи, по-прежнему могут быть изменены.

После вызова Object.seal() объект нельзя будет вернуть в прежнее состояние. Чтобы определить, вызывался ли метод Object.seal() для объекта, можно вызвать ***метод Object.isSealed()***.

Метод Object.freeze() обеспечивает еще более жесткую фиксацию объектов. Помимо того, что он делает объект нерасширяемым, а его свойства недоступными для настройки, он также делает все собственные свойства с данными доступными только для чтения. (Это не относится к свойствам объекта с методами доступа, обладающими методами записи; эти методы по-прежнему будут вызываться инструкциями присваивания.) Чтобы определить, вызывался ли метод Object.freeze() объекта, можно вызвать метод Object.isFrozen().

Важно понимать, что Object.seal() и Object.freeze() воздействуют только на объект, который им передается: они не затрагивают прототип этого объекта. Если в программе потребуется полностью зафиксировать объект, вам, вероятно, потребуется зафиксировать также объекты в цепочке прототипов.

ПОНЯТИЕ КЛАССА В JAVASCRIPT.

В JavaScript ***класс - это шаблон, описывающий свойства и методы, которыми будет обладать любой объект, созданный на основе этого класса***. Чтобы создать свой собственный класс в JavaScript, мы должны написать ***функцию-конструктор***. Например,

```
// Функция-конструктор - это обычная функция
var Class = function(p){
        console.log('My name is constructor');
        this.p = p;
}
```

А чтобы создать объект этого нового класса, мы должны вызвать его как обычную функцию, используя при этом ключевое слово ***new***. При этом ключевое слово ***this*** внутри функции-конструктора теперь будет указывать на вновь созданный объект:

```
var o = new Class('karamazov');
console.log(o); // выводит [Object object]
console.log(o.p); // выводит 'karamazov' - теперь это свойство объекта o
```

Если попытаться переменной o просто присвоить вызов функции Class() — без ключевого слова new, то никакого объекта создано не будет:

```
var o = Class('karamazov'); // эквивалентно вызову window.Class()
console.log(o); // выводит undefined, а именно то, что вернула функция Class()
console.log(window.p); // выводит 'karamazov' - теперь это свойство глобального объекта window
```

При создании функции, JavaScript автоматически создает для нее пустое ***свойство .prototype***. Любые свойства и методы, записанные в .prototype функции-конструктора станут доступными как свойства и методы объектов, созданных на основе этой функции. Это является основой для описания шаблона (класса), по которому и будут создаваться объекты. Тогда можно вызывать этот метод, как метод самого объекта. Например,

```
Class.prototype.method = function(){
        console.log('my name is .method');
}
o.method(); // работает!
```

При вызове свойства объекта, оно ищется сначала в самом объекте, и если его там не оказывается, то интерпретатор смотрит в .prototype функции-конструктора, создавшей объект. Так, при создании объекта, в нем уже существует свойство .constructor, которое указывает на функцию-конструктор, создавшую этот объект.

```
console.log(o.constructor == Class); // выводит true
```

Мы не определяли такого свойства в самом объекте. Интерпретатор, не найдя свойство .constructor в объекте, берет его из .prototype функции-конструктора, создавшей объект. Например,

```
console.log(Class.prototype.constructor == Class); // выводит true
```

Следует обратить внимание, что .prototype существует только для функции-конструктора, но не для самого объекта, созданного на его основе:

```
console.log(o.prototype); // выводит undefined
console.log(o.constructor.prototype); // выводит [Object object]
```

Доступ к .prototype функции-конструктора существует у всех объектов, в том числе и у объектов, встроенных в JavaScript, таких как строки, числа и т.п. Причем тут уже нет никаких ограничений в создании собственных свойств и методов (мы видели эти ограничения при попытке прямого присвоения свойств и методов строковой переменной — объекту, созданному через строковый литерал):

```
var s = 'karamazov';
s.constructor.prototype.tell = function(){
        console.log(this);
}
s.tell(); // теперь это не выдает ошибку, а выводит 'karamazov'
```

Задать новое свойство или метод для встроенных типов объектов можно и напрямую — через встроенную функцию-конструктор этих объектов:

```
String.prototype.tell = function(){
        console.log(this);
}
```

Свойства и методы класса

Свойства и методы класса (члены класса) могут быть ***открытыми (public)***, ***закрытыми (private)***, ***привилегированными (privileged)*** и ***статическими (static)***.

Открытые (public) свойства и методы

Открытыми называют такие свойства и методы, которые могут быть напрямую прочитаны, изменены, удалены или добавлены любым кодом, находящимся за пределами самого объекта.

Открытые свойства задаются с помощью ключевого слова .this внутри функции-конструктора:

```
var Class = function(p){
        this.p = p;
}

var o = new Class('karamazov');
console.log(o.p); // выводит 'karamazov'

o.p = 'mertas';
console.log(o.p); // выводит 'mertas'
```

Открытые методы задаются с помощью .prototype функции-конструктора:

```
Class.prototype.method = function(){
        console.log('my name is .method');
}
obj.method(); // выводит 'my name is .method'

obj.method = function(){
        console.log('my name is .method, but I am new one!');
}
obj.method(); // выводит 'my name is .method, but I am new one!'
```

Присваивая объекту obj метод .method, мы не изменяем одноименный метод в .prototype функции-конструктора, а лишь закрываем его от интерпретатора, создавая в нашем объекте новое свойство с тем же именем. Т.е. все вновь создаваемые объекты будут по-прежнему обладать стандартным методом из .prototype.

Мы можем позволить объекту вновь видеть и пользоваться методом из .prototype. Для этого нужно просто удалить свойство .method самого объекта:

```
delete o.method;
o.method(); // вновь выводит 'my name is .method'
```

Свойства и методы, заданные через .prototype функции-конструктора, не копируются во вновь создаваемые объекты. Все объекты данного класса пользуются ссылкой на одни и те же свойства и методы. Одновременно, открытые члены мы можем определять в любой точке программы, в том числе даже и после создания объекта (экземпляра) класса.

Закрытые (private) свойства и методы

Закрытые свойства и методы недоступны напрямую извне объекта. Они описываются прямо в функции-конструкторе класса и создаются при инициализации объекта. Такими свойствами обладают переменные, переданные в качестве параметров в функцию-конструктор, переменные, объявленные с помощью ключевого слова var, а также функции, объявленные как локальные внутри функции-конструктора.

```
var Class = function(p){
        var secret = p;
        var count = 3;

        var counter = function(){
                count –;
                if(count > 0){
                        return true;
                } else {
                        return false;
                }
```

```
        }
    }
```

Свойства secret, count и метод counter создаются в объекте при его инициализации. Они называются закрытыми, потому что к ним нет доступа как у кода извне объекта, так и у открытых методов самого объекта. Чтобы понять, как можно использовать эти закрытые свойства, нужно обратиться к привилегированным методам.

Привилегированные (privileged) методы

Привилегированный метод обладает доступом к закрытым свойствам и методам, а также доступен как открытым методам объекта, так и извне него. Возможно удалить или переписать привилегированный метод, но нельзя изменить его или заставить раскрыть секреты, которые он охраняет.

Привилегированный метод определяется в конструкторе с помощью ключевого слова this:

```
var Class = function(p){
        var secret = p;
        var count = 3;

        var counter = function(){
                if(count > 0){
                        count –;
                        return true;
                } else {
                        return false;
                }
        }

        this.tellSecret = function(){
                if(counter()){
                        return secret;
                } else {
                        return null;
                }
        }
}
var o = new Class('12345');

console.log(o.tellSecret()); // выводит '12345'
console.log(o.tellSecret()); // выводит '12345'
console.log(o.tellSecret()); // выводит '12345'
console.log(o.tellSecret()); // выводит null
```

```
// безуспешно пытаемся переписать закрытый метод counter,
// а на самом деле, просто создаем новый одноименный открытый метод
o.counter = function(){
        return true;
}
console.log(o.tellSecret()); // все равно выводит null
```

.tellSecret и есть привилегированный метод. Он возвращает закрытое свойство secret при первых трех вызовах, а при всех последующих начинает возвращать null. Каждый раз .tellSecret вызывает закрытый метод counter, который сам обладает доступом к закрытым свойствам объекта. Любой код имеет доступ к методу .tellSecret, но это не дает прямого доступа к закрытым членам объекта.

В отличие от открытых методов, создаваемых через .prototype, копия привилегированного метода создается в каждом создаваемом объекте, что естественно влечет за собой больший расход памяти. Закрытые и привилегированные члены создаются только в момент инициализации объекта и позже уже не могут быть изменены.

Статические (static) члены

Статические свойства и методы — это свойства и методы, привязанные к самой функции-конструктору (к самому классу). Поэтому их еще называют свойствами и методами класса. Они доступны любому коду как внутри, так и за пределами объекта:

```
var Class = function(p){
        this.p = p;
}
Class.prototype.tell = function(word){
        console.log(this.p + ' ' + word + ' ' + this.constructor.p);
        // console.log(this.p + ' ' + word + ' ' + Class.p);
}
Class.p = 'futurico';

var o = new Class('karamazov');
o.tell('love'); // выводит 'karamazov love futurico';
```

Замыкание (closure)

Закрытые и привилегированные методы возможны в JavaScript благодаря тому, что называется замыканием (closure). Замыкание — это функция, плюс все те лексические переменные из охватывающего контекста, которые она использует. Когда мы используем оператор function, мы всегда создаем не функцию, а именно замыкание. Замыкание 'помнит' значения всех переменных, которые существовали в контексте, создающем это замыкание, даже когда функция используется уже вне создавшего ее контекста.

```
var createFunc = function(param){
        var closureParam = param;
```

```
        // замыкание
        var returnedFunc = function(){console.log(closureParam);}
        return returnedFunc;
}

var f = createFunc('karamazov');
```

Теперь, если мы посмотрим на переменную f, мы увидим, что это обычная функция, в теле которой заключен параметр closureParam, который не определён нигде в окружающем f контексте и по идее должен выдавать ошибку:

```
console.log(f); // выведет: function(){console.log(closureParam);}
```

Однако ошибки не будет, функция function(){console.log(closureParam);} благодаря эффекту замыкания помнит closureParam из контекста, породившего ее:

```
f(); // выведет 'karamazov'
```

Если вспомнить описанный выше привилегированный метод .tellSecret, то теперь можно понять, как он работает. Метод помнит как закрытую функцию count(), так и закрытое свойство secret, объявленные в создающем .tellSecret контексте. При этом, когда внутри .tellSecret вызывается count(), эта последняя функция, в свою очередь, помнит использующуюся в ее теле переменную count.

Концепции ООП: инкапсуляция, наследование и полиморфизм.

Для начала, три концепции, которые должен включать в себя любой объектно-ориентированный язык программирования: инкапсуляция, наследование и полиморфизм. Знакомые слова, не так ли? Теперь простыми словами о каждой концепции:

Инкапсуляция

Инкапсуляция является одним из ключевых понятий объектно-ориентированного программирования и представляет сокрытие состояния объекта от прямого доступа извне. Инкапсуляция есть ни что иное, как реализация приватности.

По умолчанию все свойства объектов являются публичными, общедоступными, и мы к ним можем обратиться из любого места программы, например:

```
function User(pName, pAge) {
        this.name = pName;
        this.age = pAge;
        this.displayInfo = function(){
                console.log("Имя: " + this.name + "; возраст: " + this.age);
        };
};
var tom = new User("Том", 26);
tom.name=34;
console.log(tom.name);
```

Но мы можем их скрыть от доступа извне, сделав свойства локальными переменными:

```
function User (name, age) {
	this.name = name;
	var _age = age;
	this.displayInfo = function(){
		console.log("Имя: " + this.name + "; возраст: " + _age);
	};
	this.getAge = function() {
		return _age;
	}
	this.setAge = function(age) {
		if(typeof age === "number" && age >0 && age<110){
			_age = age;
		} else {
			console.log("Недопустимое значение");
		}
	}
}

var tom = new User("Том", 26);
console.log(tom._age); // undefined - _age - локальная переменная
console.log(tom.getAge()); // 26
tom.setAge(32);
console.log(tom.getAge()); // 32
tom.setAge("54"); // Недопустимое значение
```

В конструкторе User объявляется локальная переменная _age вместо свойства age. Как правило, названия локальных переменных в конструкторах начинаются со знака подчеркивания.

Для того, чтобы работать с возрастом пользователя извне, определяются два метода. Метод getAge() предназначен для получения значения переменной _age. Этот метод еще называется геттер (getter). Второй метод - setAge, который еще называется сеттер (setter), предназначен для установки значения переменной _age.

Плюсом такого подхода является то, что мы имеем больший контроль над доступом к значению _age. Например, мы можем проверить какие-то сопутствующие условия, как в данном случае проверяются тип значение (он должен представлять число), само значение (возраст не может быть меньше 0).

В JavaScript для организации инкапсуляции применяются ***замыкания***. Замыкание - это функция + все внешние переменные, которые ей доступны. Доступ к переменных в функциях осуществляется следующим образом. Все переменные внутри функции в JavaScript - это ***свойства объекта LexicalEnvironment***, так называемое лексическое окружение. Данный объект является внутренним и к нему нет доступа.

Кроме того, в специальном внутреннем свойстве у функции хранится объект ***[[scope]]***. В нём хранятся внешние для данной функции переменные. Функция сначала ищет переменные в LexicalEnvironment и, если не находит, ищет в [[scope]]. Таким образом, функция сначала ищет объявления нужных элементов внутри себя, и лишь потом обращается к внешним элементам. При этом из внешней области берётся именно текущее значение переменной. Кроме того, в JavaScript внутри функций можно объявлять и другие функции.

Рассмотрим следующий пример.

```
// Функция конструктор
var Person = function(name) {
	// Приватная функция
	var log = function(message) {
		console.log(message);
	};

	// Публичное свойство
	this.name = name;

	// Публичный метод
	this.logger = function(message) {
		log('Public ' + message);
	};
};
```

Что не так с инкапсуляцией в JavaScript? Всё просто. В примере, приведённом выше отсутствует возможность явно обозначить приватность, как в других языках программирования с помощью ключевых слов private, protected и public (пример с PHP). Естественно, подобная проблема достаточно просто решается с помощью тех же самых функций - создаётся ещё одна обёртка над кодом в виде самовызывающейся анонимной функции, например, при использовании паттерна «модуль»:

```
var Peron = (function() {
	// Приватная функция
	var log = function(message) {
		console.log(message);
	};

	var Person = function(name) {
		// Публичное свойство
		this.name = name;
	};

// Публичный метод
Person.prototype.logger = function(message) {
	log('Public ' + message);
```

```
};

// Экспорт публичной функции
return Person;
})();
```

С релизом стандарта ES2015 подобная проблема приватности была частично решена добавлением блочных областей видимости для переменных, в результате чего чистые функции, работающие только для данного конкретного конструктора можно вынести в отдельный модуль и запрашивать по необходимости, не опасаясь того, что кто-то их перезапишет.

Инкапсуляция – важная часть объектно-ориентированного программирования, чтобы данные в разных экземплярах класса были отделены друг от друга; это реализуется в JavaScript с помощью оператора this. Однако, в отличие от других объектно-ориентированных языков программирования, JavaScript не ограничивает доступ к данным внутри экземпляра класса.

Наследование

JavaScript поддерживает наследование, то позволяет нам при создании новых типов объектов при необходимости унаследовать их функционал от уже существующих. Например, у нас может быть объект User, представляющий отдельного пользователя. И также может быть объект Employee, который представляет работника. Но работник также может являться пользователем и поэтому должен иметь все его свойства и методы. Например:

```
// конструктор пользователя
function User (name, age) {
	this.name = name;
	this.age = age;
	this.go = function(){document.write(this.name + " идет");}
		this.displayInfo = function(){
		console.log("Имя: " + this.name + "; возраст: " + this.age);
	};
}
User.prototype.maxage = 110;
// конструктор работника
function Employee(name, age, comp){
	User.call(this, name, age);
	this.company = comp;
	this.displayInfo = function(){
		console.log("Имя: " + this.name + "; возраст: " + this.age + "; компания: " +
this.company );
	};
}
Employee.prototype = Object.create(User.prototype);
```

```
var tom = new User("Том", 26);
var bill = new Employee("Билл", 32, "Google");
tom.go();
bill.go();
tom.displayInfo();
bill.displayInfo();
console.log(bill.maxage);
```

Здесь в начале определяет конструктор User и к его прототипу добавляется свойство maxage. Затем определяется тип Employee.

В конструкторе Employee происходит обращение к конструктору User с помощью вызова:

```
User.call(this, name, age);
```

Передача первого параметра позволяет вызвать функцию конструктора User для объекта, создаваемого конструктором Employee. Благодаря этому все свойства и методы, определенные в конструкторе User, также переходят на объект Employee.

Кроме того, необходимо унаследовать также и прототип User. Для этого служит вызов:

```
Employee.prototype = Object.create(User.prototype);
```

Метод Object.create() позволяет создать объект прототипа User, который затем присваивается прототипу Employee. При этом при необходимости в прототипе Employee мы также можем определить дополнительные свойства и методы.

При наследовании мы можем переопределять наследуемый функционал. Например, Employee переопределяет метод displayInfo(), унаследованный от User, чтобы включить в вывод этого метода новое свойство company.

С помощью наследования вы, буквально, говорите: «У меня есть один конструктор/класс и другой конструктор/класс, который точно такой же, как и первый, кроме вот этого и вот этого». Наследование в JavaScript можно реализовать с помощью функции Object.create(), позволяющий создать новый объект с заданным прототипом.

```
// функция конструктор
var Person = function(name) {
        this.name = name + ' Doe';
};

// запись метода в прототип
Person.prototype.sayName = function() {
        console.log(this.name);
};

// Вызов конструктора родителя внутри дочернего
// конструктора для записи всех свойств
```

```
var GreatPerson = function(name, phrase) {
        Person.apply(this, arguments);
        this.phrase = phrase;
};

// Перезапись прототипа дочернего конструктора
GreatPerson.prototype = Object.create(Person.prototype);

GreatPerson.prototype.sayPhrase = function() {
        console.log(this.name + ' says: "' + this.phrase + '"');
};

// создание нового объекта
var john = new Person('John');
var jane = new GreatPerson('Jane', 'Hello, World!');

john.sayName(); // John Doe
jane.sayName(); // Jane Doe
jane.sayPhrase(); // Jane Doe says: "Hello, World!"
```

Да в чём вообще кроется разница? Мы же выполнили одни и те же действия, просто с разным синтаксисом! Чтобы понять, почему JavaScript другой представьте себе семейство птичек: дедушка попугай, отец попугай и сын попугай — все попугаи! Совершенно очевидно, что если у деда попугая вырастет ещё одна лапка, то это ни коем образом не повлияет на отца и сына. То есть попугай, родившийся с двумя лапами, так и останется до конца своей жизни с двумя лапами, в независимости от того, что случилось с любым из его предков. Подобным образом можно представить себе классическое наследование.

С прототипным наследование ситуация абсолютно противоположная. Дед-попугай отрастил себе третью лапку, и она автоматически появилась у отца и сына (прототипное наследование против природы). Сложившееся положение вещей крайне не понравилось отцу попугаю, и он решил, что и попугаем-то больше быть не хочет и стал орлом (с тремя лапами). Как вы уже, наверное, догадались сын попугай уже больше не попугай, а настоящий орёл (но опять же с тремя лапами). Три лапы слишком много для сына и он решает отказаться от одной (теперь у нас есть обычный орёл с двумя лапами).

Но и это ещё не всё! Единственный оставшийся попугай (дедушка) решил, что трёх лап мало и приобрёл себе ещё одну, а также решил стать ласточкой. В результаты из обычного семейства попугаев мы получили: деда ласточку с четырьмя лапами, отца орла с тремя лапами, сына орла с двумя лапами. Вот она вся суть прототипного наследования. Запутанно? Тогда перейдём к коду:

```
// Дед попугай с двумя лапами
var ParrotGrandfather = function() {};
ParrotGrandfather.prototype = {
  species: 'Parrot',
  paws: 2
```

```
};

// Отец попугай унаследовал всё у деда
var ParrotFather = function() {};
ParrotFather.prototype = Object.create(ParrotGrandfather.prototype);

// Сын попугай унаследовал всё у отца
var ParrotSon = function() {};
ParrotSon.prototype = Object.create(ParrotFather.prototype);

var grandfather = new ParrotGrandfather();
var father = new ParrotFather()
var son = new ParrotSon();

console.log(grandfather.species, father.species, son.species);
// Parrot Parrot Parrot - все попугаи!
console.log(grandfather.paws, father.paws, son.paws);
// 2 2 2 - у каждого по 2 лапы

// Дед меняет количество лап
ParrotGrandfather.prototype.paws++;
console.log(grandfather.paws, father.paws, son.paws);
// 3 3 3 - у каждого теперь по 3 лапы

// Отец меняет вид
ParrotFather.prototype.species = 'eagle';
console.log(grandfather.species, father.species, son.species);
// Parrot eagle eagle - дед остался попугаем, отец и сын стали орлами

// Сын уменьшил количество лап
ParrotSon.prototype.paws--;
console.log(grandfather.paws, father.paws, son.paws);
// 3 3 2 - дед и отец остались при своих трёх лапах

// Дед решил стать чайкой
ParrotGrandfather.prototype.species = 'seagull';
console.log(grandfather.species, father.species, son.species);
// seagull eagle eagle - дед чайка, отец и сын орлы
```

Вывод из всего выше перечисленного: в JavaScript прототипное наследование «динамическое», можно изменять всё налету, классическое же наследование подобным похвастаться не может: всё, что

вы объявили в одном классе, останется там навсегда. Грубо говоря, классическое представление наследования предполагает наличие определённой статической схемы, по которой будет строиться каждый объект данного класса. В прототипном наследовании мы имеем дело не со схемой, а с живым, постоянно развивающимся организмом, который со временем изменяется и принимает ту форму, которая нам нужна (и это прекрасно).

Полиморфизм

Полиморфизм проще всего постичь на примере встроенных конструкторов (String, Array, Object…). Вот если вас спросят: “Чем число 42 отличается от массива [4, 2] и что у них общего?”, чтобы вы ответили? Наверняка, вы были начали рассказывать про примитивы и объекты, чем они отличаются, что можно делать с теми и другими, на вопрос про отличия. Но чем они похожи друг на друга? Абсолютно разные же типы данных! Но, очевидно, что они разделяют определённую часть методов, например, метод toString, унаследованный от Object. Это уже полиморфизм? Ещё нет, но мы уже близко. Метод toString можно весьма успешно переназначить, во-первых, в прототипе функции конструктора, и, во-вторых, сразу же для данного конкретного объекта.

```
// Наш собственный конструктор
var Person = function(name) {
  this.name = name;
};

// Переназначение метода toString для всех объектов,
// созданных с помощью данного конструктора
Person.prototype.toString = function() {
  return 'Person ' + this.name;
};

var john = new Person('John');

// Два массива, второй абсолютно обычный,
// для первого переназначен метод toString
var arr1 = [4, 2];
var arr2 = [5, 3];
arr1.toString = function() {
  return 'Array ' + this.reduce(function(result, item) {
    return result + '' + item;
  });
};

// В итоге
console.log(john.toString()); // Person John
console.log(arr1.toString()); // Array 42
```

```
console.log(arr2.toString()); // 5,3
```

Что происходит? При использовании метода toString на каждом из объектов происходит проверка того, какой метод нужно выбрать. Проверка ведётся следующим образом: изначально проверяется существует ли нужное свойство на самом объекте, если существует, то используется именно оно, если же нет, то проверка продолжается — на наличие свойства проверяется прототип, потом прототип прототипа, потом прототип прототипа прототипа… и так, пока не дойдём до самого конца — null (null является прототипом для Object). Таким образом, полиморфизм отвечает за то, чей метод вызвать.

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Самый лучший общедоступный ресурс с точки зрения простоты изложения материала – это «Современный учебник по Java Script», доступный по ссылке https://learn.javascript.ru/books .

Еще один учебник Антон Кармазин «Выразительный JavaScript», доступен по ссылке https://www.gitbook.com/book/karmazzin/eloquentjavascript_ru/details .

Руководство, которое можно использовать в качестве карманного справочника - http://bonsaiden.github.io/JavaScript-Garden/ru/ .

Данных ресурсов достаточно, чтобы получить начальные навыки владения языком программирования.

Печатные издания:

1. Вайсфельд М. Объектно-ориентированное мышление. – СПб.: Питер, 2014. – 304 с.
2. Стефанов С. JavaScript. Шаблоны. – Пер. с англ. – СПб.: Символ-Плюс, 2011. – 272 с.
3. Флэнаган Д. JavaScript. Подробное руководство. – Пер. с англ. – СПб: Символ-Плюс, 2008. – 992 с.
4. Закас Н. JavaScript для профессиональных веб-разработчиков / (Пер. с англ. А. Лютича). – СПб.: Питер, 2015. – 960с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Понятие класса. Как класс интерпретируется в языке JavaScript?
2. Принципы ООП. Инкапсуляция. Приведите пример.
3. Принципы ООП. Наследование. Приведите пример.
4. Принципы ООП. Полиморфизм. Приведите пример.
5. Как можно определить класс в JavaScript?
6. Какие особенности имеются у классов без конструкторов?
7. Что такое конструктор класса? Приведите пример.